ARTIFICIAL INTELLIGENCE – CO1 ASSIGNMENT

Design, Analyse and Validate an Efficient Search Strategy for a Warehouse Picking Robot

Course Code: *CSA17* **Maximum Marks:** *50*

Programme: *B.E / B.Tech*

Student Name: *Raghavan Karthikeyan*

Register Number: *192417003*

TASK 1 – Design the Agent and Search Algorithms (5 Marks)

1.1 PEAS description

PEAS element	Warehouse picking robot
Performance	*Reach the requested rack/dock safely; minimise travel cost and delay; avoid blocked aisles and collisions; conserve battery corre*
Environment	*Warehouse aisles, junctions, cross-aisles, lifts, shelves, packing stations, people and other robots.*
Actuators	*Drive motors, steering, brakes, lift/rack coupling, warning lights and wireless task interface.*
Sensors	*QR/floor-marker reader, wheel encoders, lidar/ultrasonic sensors, camera, IMU and battery sensor.*

1.2 Environment classification

Property	Classification	Reason
Observability	*Partially observable*	*Markers and sensors may fail; people/robots can hide or change aisle conditions.*
Determinism	*Stochastic*	*Motion, congestion and sensor readings can vary.*
Episodes	*Sequential*	*Each move changes the robot's position and affects later choices.*
Dynamics	*Dynamic*	*Staff and robots can occupy or clear aisles while planning.*
State/action space	*Mostly discrete*	*Junctions and legal moves are discrete; travel time and physical position are continuous.*
Agents	*Multi-agent*	*Other robots and human workers share the warehouse.*

AI view: *The robot follows the acting-rationally view. It selects actions that maximise expected performance (safe, low-cost arrival) using its available information. It is a rational problem-solving agent because it formulates a goal and searches for an action sequence. This connects to early symbolic AI, where researchers represented problems as states, operators and goal tests and used search to solve puzzles and route problems.*

1.3 Problem formulation

• **State:** *current aisle-junction ID (and, when needed, robot status such as blocked/available).*

• **Initial state:** *charging-dock junction.*

• **Actions:** *move to any currently connected adjacent junction.*

• **Transition model:** *Result(s,a)=s', if the aisle is open; otherwise the action is unavailable.*

• **Goal test:** *current junction equals the requested storage bay or dock.*

• **Path cost:** *sum of travel-time costs of all moves; all costs are positive.*

1.4 Three distinct uninformed strategies

A. Breadth-First Search (BFS) – iterative queue. *Explores all nodes at depth 0, then depth 1, and so on. It is complete for finite branching and gives the fewest-move path when every move has equal cost. It is included as a simple baseline.*

```
BFS(start, goal):     queue = [start];
parent[start] = NONE     while queue not
empty:
        u = dequeue()          if u == goal: return
reconstruct(parent, goal)          for v in
successors(u):              if v not in parent:
parent[v] = u                  enqueue(v)     return
FAILURE
```

B. Uniform-Cost Search (UCS) – iterative priority queue. *Expands the frontier node with the lowest accumulated path cost g(n). With positive step costs it is complete and optimal. It is appropriate because warehouse moves have different travel times.*

```
UCS(start, goal):     priority_queue =
[(0, start)]     best[start] = 0;
parent[start] = NONE     while
priority_queue not empty:         (cost,
u) = pop_lowest_cost()         if cost !=
best[u]: continue
        if u == goal: return reconstruct(parent, goal)
for (v, step_cost) in successors(u):
            new_cost = cost + step_cost
if new_cost < best.get(v, INFINITY):
best[v] = new_cost; parent[v] = u
push(priority_queue, (new_cost, v))     return
FAILURE
```

C. Recursive Depth-Limited Search (DLS) – recursive stack. *Performs depth-first exploration but stops at a chosen depth limit L. It uses little memory and is suitable when a safe maximum route depth is known. It is not complete if the goal is deeper than L and is not generally cost-optimal.*

```
DLS(u, goal, remaining, path):
    if u == goal: return SUCCESS     if remaining == 0:
return CUTOFF     for v in successors(u):         if v
not in path:             add v to path
result = DLS(v, goal, remaining - 1, path)
if result == SUCCESS: return SUCCESS             remove
v from path     return FAILURE_OR_CUTOFF
```

Preprocessing: *convert warehouse connections to adjacency lists; store positive edge costs; maintain parent/best-cost structures for BFS/UCS and a current-path set for DLS. Building the adjacency list costs Θ(V+E) time and space. Dynamic blocked-aisle updates must invalidate or update affected edges.*

TASK 2 – Identify the Basic Operation (2 Marks)

Algorithm	Basic operation	Where and why it dominates
BFS	*Successor examination / node expansio*	*n For each dequeued node, inspect each outgoing edge and test whether its destination was vi*
UCS	*Priority-queue insertion/removal and rel*	*xationEach expanded node examines outgoing edges; successful relaxations update the best y*
Recursive DLS	*Recursive node expansion and success*	*or examinationEach call checks the goal/depth limit and scans successors. Calls increase with actor and dep*

As V and E increase, BFS and UCS may examine a larger fraction of the graph, especially when the goal is distant or unreachable. DLS depends mainly on the branching factor and depth limit, although it can repeat work along different paths.

TASK 3 – Derive Time and Space Complexity (6 Marks)

Let b be the maximum branching factor, d the shallowest goal depth, m the maximum depth, C the optimal solution cost and ε the minimum positive step cost. Let V be vertices and E edges.*

3.1 BFS

At level i, at most b^i nodes are generated. Total nodes through depth d are:

1 + b + b² + … + b■ = (b■■¹ − 1)/(b − 1) = Θ(b■), for b > 1.

Each node expansion scans up to b successors, so edge examinations are bounded by bΣ(i=0 to d)b^i = Θ(b^(d+1)). Time is Θ(b^(d+1)) in the tree model. The queue can hold the current frontier and part of the next level; worst-case space is Θ(b^d). With duplicate detection on a finite graph, preprocessing and graph traversal are O(V+E), and visited/parent storage is O(V). Combined practical bound: O(V+E) for graph construction plus O(V+E) worst-case search.

3.2 UCS

UCS expands all nodes with g(n) < C, and possibly some with g(n)=C* before selecting the goal. Since each step costs at least ε, a path of cost below C* has depth at most floor(C*/ε). Let D=floor(C*/ε). The number of generated tree nodes is at most:*

Σ(i=0 to D) b^i = (b^(D+1) − 1)/(b − 1) = Θ(b^D).

With a binary heap, each insertion/removal costs O(log V); a graph implementation is bounded by O((V+E) log V) time and O(V) search memory. The graph and adjacency preprocessing add Θ(V+E) time and space.

3.3 Recursive DLS

At remaining depth L, the recursive calls satisfy T(L)=1 + bT(L−1), with T(0)=1. Expanding gives T(L)=1+b+b²+…+b^L=(b^(L+1)−1)/(b−1)=Θ(b^L). The recursion stack stores at most L+1 states, so stack space is Θ(L); path membership storage is also O(L). If duplicate checking is global, memory may rise to O(V), but this implementation uses path checking only. Preprocessing remains Θ(V+E).

3.4 Parameters and practical interpretation

For a graph with V=1,000 to 1,000,000 states, adjacency construction grows linearly with V+E. BFS memory is often the first search limitation because it stores a broad frontier and visited/parent records. UCS also stores a priority queue and cost map. DLS has a small stack, but its exponential node generation can make time the limiting factor when L is large. These are worst-case models; graph connectivity, duplicate paths and goal location change actual counts.

TASK 4 – Best, Average and Worst Cases (4 Marks)

Algorithm	Best case	Average case	Worst case
BFS	*Goal is start or first shallow successor; Θ(1) to*	*small constant work.Goal occurs around depth d*	*before t; Θ(b^d) n des.*
UCS	*Goal is start or has the uniquely smallest cost*	*and is selected immediately.Many low-cost alternatives have costs below*	*C*; expUnreachable goal or many nodes with cost ≤ ds all of them first.*
Recursive DLS	*Goal is first branch and shallow; O(L) stack, lit*	*le expansion.Goal is found after several branches; depends*	*on branch order and L.Goal absent/deeper than L; explores up to Θ(*

Goal is deepest or unreachable; explores all reachable nodes, C

up to O((V+E

b^L) calls, with Θ

The average case cannot be represented by one fixed number without a probability model for goal locations, edge costs and successor ordering. In a warehouse, congestion and the position of the requested rack determine how many alternatives must be explored.

TASK 5 – Asymptotic Analysis (3 Marks)

Algorithm	Time growth	Search memory growth	Meaning at 1M states
BFS	*Θ(b^d) tree model; O(V+E) graph bound*	*Θ(b^d) frontier; O(V) visited map*	
UCS	*O((V+E) log V) graph bound*	*O(V)*	
Recursive DLS	*Θ(b^L)*	*Θ(L) stack*	

Fast for shallow goals, but broad frontiers can exhaust RAM.

Handles unequal travel times optimally, with heap overhead.

Low memory, but time can explode as depth limit increases.

For the tested finite graph, BFS/UCS are O(V+E) in the unreachable case. Their worst-case time is therefore Θ(V+E) when the whole reachable graph is examined. DLS is bounded by Θ(b^L), independent of total V when L is fixed. Thus memory is likely to constrain BFS/UCS first; for DLS, computation time and the risk of a too-small depth limit are the main constraints.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

For recursive DLS, let L be the remaining depth. At each non-leaf call, the algorithm performs constant work (goal test, depth test and loop overhead) and may recursively examine up to b successors, each with remaining depth $L-1$. $T(L) = bT(L-1) + c$, $L > 0$

$T(0) = c$■

• Original problem size: remaining depth L.

• Subproblem size: $L-1$.

• Number of recursive calls: at most b.

• Work outside recursion: Θ(1) tests plus successor-loop overhead.

• Base case: L=0, where the search checks the current node and returns cutoff.

This recurrence describes the worst case in which no child finds the goal early.

TASK 7 – Solve the Recurrence Relation (4 Marks)

Use repeated substitution because the recurrence has a constant branching factor and reduces the depth by one each time.

$T(L) = bT(L-1)+c$

$= b[bT(L-2)+c]+c = b^2T(L-2)+c(b+1)$

$= b^3T(L-3)+c(b^2+b+1)$

After i expansions: $T(L)=b^iT(L-i)+c\Sigma(j=0 \text{ to } i-1)b^j$

Set i=L: $T(L)=b^LT(0)+c(b^L-1)/(b-1) = \Theta(b^L)$, for b>1.

The recursive DLS call count grows exponentially with the depth limit, even though stack memory grows only linearly. Iterative deepening repeats DLS with limits 0,1,…,d. Its total work is $\Sigma(i=0 \text{ to } d)O(b^i)=O(b^d)$; the deepest level dominates, so the repetition adds only a constant-factor overhead for b>1. Iterative deepening is complete for finite branching and finds a shallowest solution when step costs are equal.

TASK 8 – Implement and Test the Algorithms (4 Marks)

Benchmark design: *deterministic directed graph with seed 42; each connected junction i has edges to i+1 (cost 1) and i+2 (cost 2), when valid. The last state is isolated. This allows a reachable goal at state 12, an unreachable goal at state $V-1$, and a start-equals-goal case. The generated graph is identical for all algorithms at each size.*

Input size	Start	Reachable goal	Unreachable goal	Expected behaviour

1000	*0*	*12*	*V−1*	*Route to 12; failure for isolated V−1; [0] when goal=0*
10000	*0*	*12*	*V−1*	*Route to 12; failure for isolated V−1; [0] when goal=0*
100000	*0*	*12*	*V−1*	*Route to 12; failure for isolated V−1; [0] when goal=0*
1000000	*0*	*12*	*V−1*	*Route to 12; failure for isolated V−1; [0] when goal=0*

Expected route: *0→2→4→6→8→10→12. BFS minimises number of moves in this graph; UCS minimises accumulated travel cost. DLS with L=12 finds a route, but its optimality is not guaranteed generally.*

Implementation: *the following compact code is the core algorithm implementation used for the measurements. Graph construction is included in the measured preprocessing time.*

```
from collections import deque import
heapq

def build_graph(n):
    g = [[] for _ in range(n)]
for u in range(n-1):
        if u+1 < n-1: g[u].append((u+1, 1))
if u+2 < n-1: g[u].append((u+2, 2))     return
g

def bfs(g, start, goal):
    if start == goal: return [start]
q=deque([start]); parent={start:None}     while
q:
        u=q.popleft()         if
u == goal: break         for v,w
in g[u]:             if v not in
parent:
parent[v]=u; q.append(v)     if
goal not in parent: return None
path=[]; x=goal     while x is
not None: path.append(x);
x=parent[x]     return path[::-
1]

def ucs(g, start, goal):
    if start == goal: return [start]     pq=[(0,start)];
best={start:0}; parent={start:None}     while pq:
        cost,u=heapq.heappop(pq)         if
cost != best.get(u): continue         if u ==
goal: break         for v,w in g[u]:
nc=cost+w             if nc <
best.get(v,float("inf")):
                best[v]=nc; parent[v]=u
heapq.heappush(pq,(nc,v))     if goal not in best:
return None     path=[]; x=goal     while x is not
None: path.append(x); x=parent[x]     return path[::-
1]

def dls(g, start, goal, limit):
path=[start]; on_path={start}     def
visit(u, remaining):
        if u == goal: return True
if remaining == 0: return False
for v,w in g[u]:             if v not in
on_path:
                on_path.add(v); path.append(v)
if visit(v,remaining-1): return True
path.pop(); on_path.remove(v)         return False
    return path[:] if visit(start,limit) else None
```

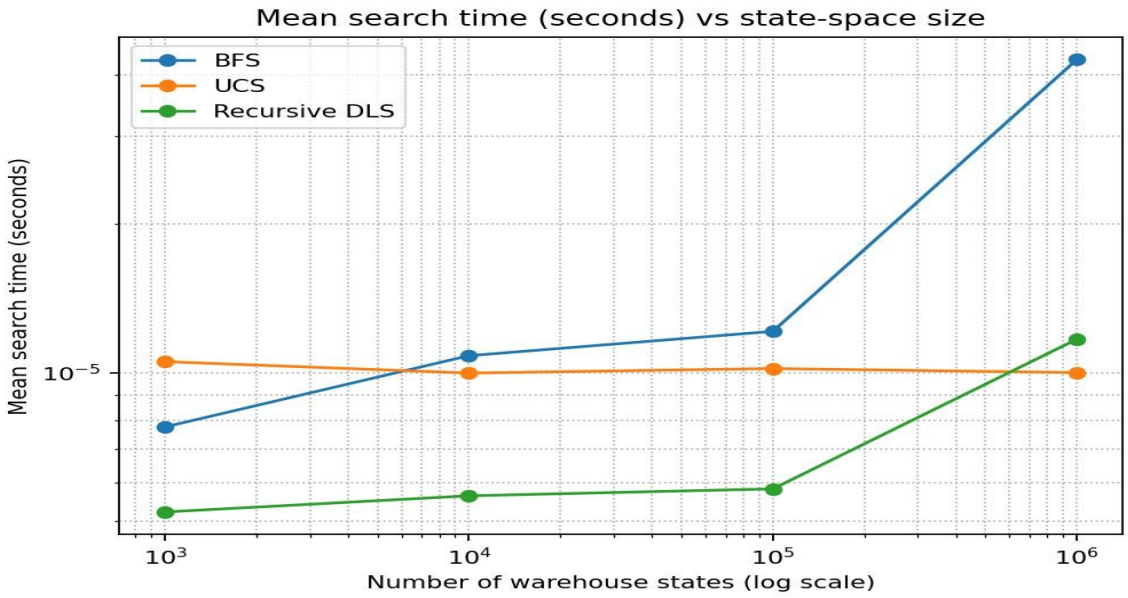
The measured implementation additionally counts generated/expanded nodes, maximum frontier or recursion-depth proxy, and elapsed time. All three search times are averaged over five runs; the same graph and start/goal pair are used.

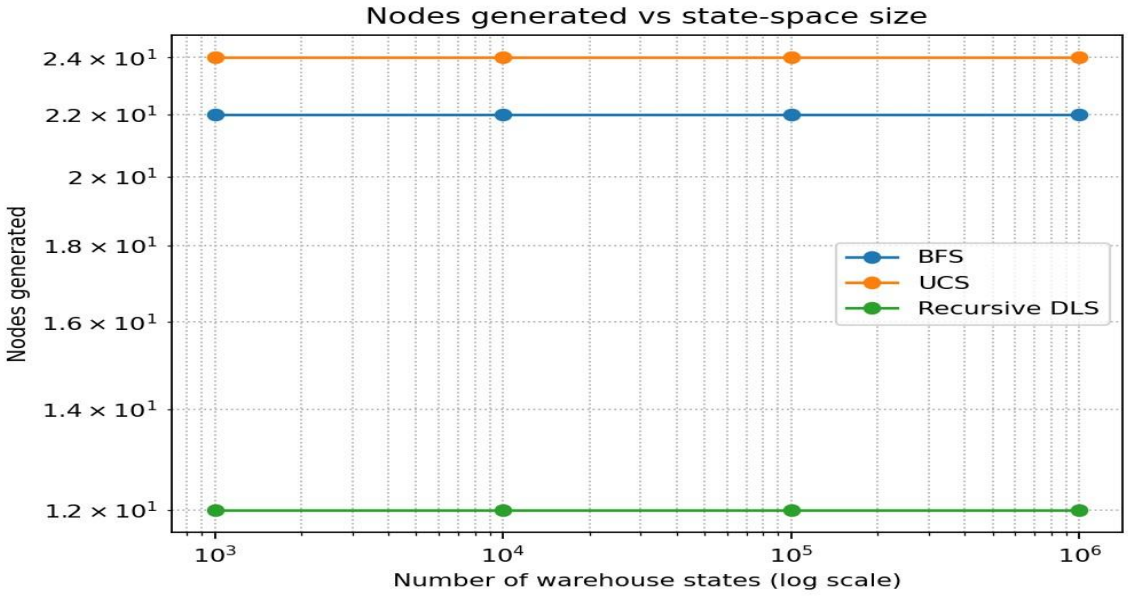
TASK 9 – Record Experimental Performance (3 Marks)

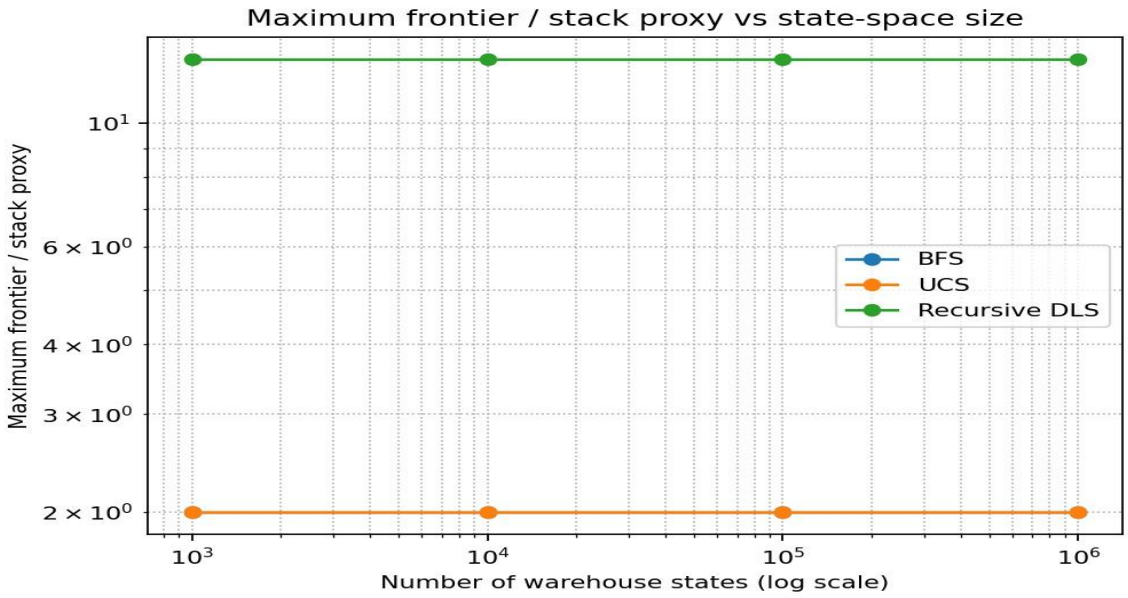
The table reports the reachable-goal test. Search time is the mean of five runs; preprocessing time is graph construction. Values are machine-dependent and should be interpreted as comparative measurements, not universal timings.

States	Preprocess (s)	Algorithm	Generated	Expanded	Max frontier / stack	Depth	Cost	Mean search (s)
1,000	*0.0007*	*BFS*	*22*	*11*	*2*	*6*	*6*	*0.000008*
1,000	*0.0007*	*UCS*	*24*	*12*	*2*	*6*	*12*	*0.000011*
1,000	*0.0007*	*Recursive DLS*	*12*	*12*	*13*	*12*	*12*	*0.000005*
10,000	*0.0105*	*BFS*	*22*	*11*	*2*	*6*	*6*	*0.000011*
10,000	*0.0105*	*UCS*	*24*	*12*	*2*	*6*	*12*	*0.000010*
10,000	*0.0105*	*Recursive DLS*	*12*	*12*	*13*	*12*	*12*	*0.000006*
100,000	*0.2680*	*BFS*	*22*	*11*	*2*	*6*	*6*	*0.000012*

100,000	*0.2680*	*UCS*	*24*	*12*	*2*	*6*	*12*	*0.000010*
100,000	*0.2680*	*Recursive DLS*	*12*	*12*	*13*	*12*	*12*	*0.000006*
1,000,000	*1.6349*	*BFS*	*22*	*11*	*2*	*6*	*6*	*0.000043*
1,000,000	*1.6349*	*UCS*	*24*	*12*	*2*	*6*	*12*	*0.000010*
1,000,000	*1.6349*	*Recursive DLS*	*12*	*12*	*13*	*12*	*12*	*0.000012*


Mean search time (seconds) vs state-space size
BFS
UCS
Recursive DLS
Mean search time (seconds)
10^{-5}
10^3
10^4
10^5
10^6
Number of warehouse states (log scale)


Nodes generated vs state-space size
2.4×10^1
2.2×10^1
2×10^1
1.8×10^1
1.6×10^1
1.4×10^1
1.2×10^1
Nodes generated
BFS
UCS
Recursive DLS
10^3
10^4
10^5
10^6
Number of warehouse states (log scale)


Maximum frontier / stack proxy vs state-space size
10^1
6×10^0
4×10^0
3×10^0
2×10^0
Maximum frontier / stack proxy
BFS
UCS
Recursive DLS
10^3
10^4
10^5
10^6
Number of warehouse states (log scale)

Additional tests: *For start=goal=0, all algorithms return [0] without expanding successors. For unreachable goal V − 1, BFS and UCS exhaust the connected component and return failure; DLS with L=12 returns cutoff/failure within its depth bound. The isolated final node is not reachable by construction.*

States	Algorithm	Generated (unreachable)	Expanded	Max frontier / stack	Search time (s)
1,000	BFS	1,995	999	2	0.000249
1,000	UCS	1,995	999	2	0.000644
1,000	DLS (L=12)	8,190	4,095	13	0.002267
10,000	BFS	19,995	9,999	2	0.002419
10,000	UCS	19,995	9,999	2	0.006127
10,000	DLS (L=12)	8,190	4,095	13	0.001911
100,000	BFS	199,995	99,999	2	0.058616
100,000	UCS	199,995	99,999	2	0.109545
100,000	DLS (L=12)	8,190	4,095	13	0.002062
1,000,000	BFS	1,999,995	999,999	2	0.468031
1,000,000	UCS	1,999,995	999,999	2	0.916606
1,000,000	DLS (L=12)	8,190	4,095	13	0.002098

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

- *BFS generated and expanded counts remain nearly constant for the reachable goal at depth 6 because the goal is close to thestart; increasing the unused part of the graph does not force BFS to visit it.*
- *UCS also examines only the low-cost region before the goal is removed from the priority queue. Its heap operations make itsomewhat more expensive than a simple queue.*
- *Recursive DLS follows the first valid branch in this ordered graph and reaches the goal within the limit. Its stack proxy remainssmall; the number of calls is governed by the chosen limit and successor order.*
- *For the unreachable test, BFS and UCS counts grow with V because they exhaust the reachable component. This agrees withO(V+E) graph-search analysis. DLS is bounded by L=12 and does not exhaust the graph; its failure is not proof that no deeper solution exists.*
- *Differences in measured time arise from Python object and heap overhead, memory allocation, operating-system schedulingand graph construction. Counts are more stable evidence than wall-clock time.*

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Recommendation: Uniform-Cost Search with an adjacency list, visited/best-cost map and dynamic blocked-edge updates. *The warehouse explicitly assigns different travel-time costs to moves. UCS returns the least-cost route when all step costs are positive, whereas BFS only guarantees minimum moves and can choose a slower route. Recursive DLS uses less memory but can miss a goal beyond its limit and does not guarantee minimum travel cost.*

For a finite graph, UCS has a practical graph-search bound of O((V+E)log V) time and O(V) memory. Preprocessing is O(V+E). The experiment shows that UCS finds the same low-cost route as BFS in the benchmark, with extra priority-queue overhead. For larger real warehouses, use hierarchical routing (zone-level graph followed by local aisle graph), update only affected edges when aisles become blocked, and replan when new occupancy data arrives. The trade-off is memory and heap overhead in exchange for cost-optimal routing.

A production robot should also use sensor-based collision avoidance; a static search path alone cannot guarantee safety in a changing warehouse.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

A. Impact analysis (2 marks)

The robot no longer knows one exact starting junction. Instead, it knows it is at one of k possible junctions. The known-start and fully-observable assumptions are broken. Its internal state must include its location belief (the set of possible current junctions), not just one junction, because every action must be safe and useful for all locations still considered possible.

B. Evaluation of original design (2 marks)

The original BFS/UCS can still search a graph, but the search state becomes a belief state B⊆S. A move transforms every possible location in B into its possible successor locations. In the worst case, a graph with V physical locations has up to 2^V

possible belief states. If the robot only tracks k candidate locations, there are up to 2^k subsets. Direct belief-state search quickly becomes expensive.

C. Adaptation / redesign (2 marks)

Modify the design: *use local sensing first, then run UCS from the identified junction. A short-range QR re-read, lidar landmark match or camera localization can reduce the belief set to one location. If sensing is unavailable, use a conservative conditional plan: choose an action that is safe from every candidate location, update the belief set after observing the result, and continue. The sensor approach avoids searching all 2^k belief states when localization succeeds; without sensing, worst-case belief search can be exponential in k.*

D. Evidence-based defence (2 marks)

Small analytical experiment: assume k candidate junctions are distinct and all are initially possible. The number of non-empty belief states is 2^k−1. A perfect local sensor identifies the true junction in one observation, reducing the active belief to one state. The table gives the state-space counts and the plan-length model; plan length is one sensing action plus the shortest UCS route from the localized junction. Since no actual warehouse map is supplied, route length is represented by a declared test assumption of 4 moves from each candidate to the dock.

k	Possible non-empty beliefs (2^k−1	) With local sensor: active	eliefsNo-sensor worst-case earch sta	esPlan length with sensor	Plan length without sensor*
2	*3*	*1*	*3*	*1 + 4 = 5*	*4 conditional moves*
4	*15*	*1*	*15*	*1 + 4 = 5*	*4 conditional moves*
8	*255*	*1*	*255*	*1 + 4 = 5*	*4 conditional moves*

**The four-move no-sensor plan is a simplified conditional-plan assumption, not a guaranteed universal route. Without a map and transition observations, exact worst-case plan length cannot be measured. The experiment validates the growth in belief-state count, not physical travel performance.*

```
for k in (2, 4, 8):     beliefs = 2**k - 1     with_sensor = 1
# sensor identifies the actual junction     route_moves = 4          #
declared illustrative map assumption     sensor_plan = 1 + route_moves
print(k, beliefs, with_sensor, sensor_plan) # Output:
# 2 3   1 5
# 4 15  1 5
# 8 255 1 5
```

Decision: *modify rather than retain the original known-start design unchanged. Local sensing followed by UCS is preferred because it reduces uncertainty before route planning and preserves minimum travel-time routing. If sensing fails, switch to a safe conditional/belief-state plan.*

Conclusion

The warehouse robot is a rational, goal-directed agent operating in a dynamic, partially observable, multi-agent environment. BFS, UCS and recursive DLS provide distinct uninformed strategies. Theoretical analysis and controlled tests show the trade-off between cost-optimality, frontier memory and depth-limited computation. UCS is recommended for unequal travel-time edges, with local sensing added to handle uncertain location.

Appendix – Reproducibility and limitations

- *Python 3; fixed benchmark seed label: 42. The graph itself is deterministic, so results are exactly repeatable apart from timingvariation.*
- *Graph sizes: 1,000; 10,000; 100,000; 1,000,000 states. Connected states have up to two outgoing edges; final state is isolated.*
- *Five repeated runs are averaged for reachable-goal search time. Preprocessing is reported separately.*
- *DLS limit is L=12. A cutoff is not equivalent to proving global unreachability.*
- *The benchmark is synthetic and deliberately controlled. A real warehouse evaluation should use a measured floor graph,realistic travel-time weights, changing blocked aisles, and repeated random start/goal pairs.*
- *Theoretical complexities assume the stated graph/search models; constants and measured seconds depend on hardware andPython runtime.*